YUKTHA

Deepfake Defense System

Multi-modal AI for authentic digital content verification

Gaddam Koushik - 22071A6916
Paitha Kommineni - 22071A6934
K.N.R. Rishik - 22071A6938
Konala Sri Sai Pranav - 23075A6901

Guide - Mrs. V Manjula

Try Pitch

Agenda

1. Abstract
2. Summary of Literature Review
3. Proposed System Architecture
4. System Requirements
5. System Architecture Diagram
6. UML Diagrams
7. Core Modules
8. Implementation
9. Current Project Status

Try Pitch

Abstract

Deepfake media generated using advanced generative models poses a serious threat to digital trust, enabling misinformation, identity fraud, and social manipulation. Existing deepfake detection approaches are largely model-centric, focusing on either audio or video modalities in isolation, often relying on naive fusion strategies and lacking explainability and deployment readiness. In this work, we propose a lean, multimodal deepfake detection system that integrates spatial, temporal, audio, frequency, physiological, and lip-sync cues through a learning-based decision-level fusion framework. The proposed system processes audio and video using independent pipelines and performs confidence-aware fusion to improve robustness against partial and fully synthetic manipulations. Lightweight audio–visual lip-sync verification is employed by default, with optional escalation to advanced phoneme-level models such as SyncNet or AV-HuBERT under low-confidence, talking-head scenarios. Additionally, the system provides explainable outputs using visual and audio heatmaps, temporal plots, and confidence breakdowns, enabling forensic interpretability. Designed with real-world deployment in mind, the system adopts an API-driven architecture suitable for web, mobile, and enterprise use. This work bridges the gap between academic deepfake detection models and practical, trustworthy detection systems.

Try Pitch

Summary of Literature Review

Video Deepfake Detection Approaches

State-of-the-Art Architectures:

- **XceptionNet:** Achieves 95% accuracy on FaceForensics++ through frequency domain analysis
- **MesoNet (MesoInception-4):** 84% on DeepFake TIMIT with lightweight design
- **Capsule Networks:** 96.6% accuracy by capturing spatial inconsistencies
- **ResNeXt-50 + LSTM:** 94.3% on Celeb-DF v2 (hybrid spatial-temporal approach)
- **EfficientNet-B4 + LSTM:** 93.8% on DFDC dataset

Critical Insight: CNN-only approaches rely heavily on global frame features and often miss localized manipulation artifacts. The most robust detectors combine:

- Spatial analysis (CNN extracting pixel-level inconsistencies)
- Temporal analysis (LSTM/RNN modeling frame-to-frame dynamics)
- Attention mechanisms (guiding focus to artifact-prone regions)

Audio Deepfake Detection Approaches

Feature Extraction Methods:

- **MFCC (Mel-Frequency Cepstral Coefficients):** Mimics human auditory perception; achieved 98.83% accuracy with SVM on clean datasets
- **Mel-Spectrograms:** Better captures frequency variations; used with VGG-16 for 93% accuracy on Fake-or-Real dataset
- **Spectrogram Analysis:** Reveals temporal inconsistencies in synthetic voice generation

Effective Architectures:

- **CNN-RNN Hybrid:** CNN extracts spectral features from spectrograms; RNN/LSTM models temporal sequences
- **BiLSTM Networks:** Bidirectional temporal analysis captures both forward and backward dependencies
- **Attention-Based Models:** Focus on frequency bands most susceptible to manipulation

Try Pitch

Summary of Literature Review

Multi-Modal Deepfake Detection

Fusion Strategy	Description	Advantage	Use Case
Simple, modular; easy interpretation	Independent classifiers for each modality; decisions averaged at output	Late Fusion	Lightweight systems
Balanced speed/accuracy	Features concatenated before final classification layer	Mid Fusion	Standard deployments
Superior performance; captures cross-modal patterns	Raw features concatenated early; transformer processes combined representation	Early Fusion	High-precision applications

Key Finding: Early Fusion with Transformer architectures outperforms late fusion by 3-5% on multimodal datasets (FakeAVCeleb, DDL-AV)

Try Pitch

Proposed System Architecture

🔴 Gap 1: Lack of True Multi-Modal Fusion

Gap:

Most works use only audio or video. Even audio-visual methods rely on feature concatenation or simple averaging, with no learning-based trust between modalities.

Our Approach:

Independent audio and video pipelines with decision-level, learning-based fusion and confidence calibration for adaptive modality weighting.

🔴 Gap 2: Weak Lip-Sync Analysis

Gap:

Lip-sync and phoneme-lip alignment are largely ignored, especially for talking-head deepfakes.

Our Approach:

Lightweight lip motion vs audio energy correlation, with optional SyncNet / AV-HuBERT for low-confidence cases.

Try Pitch

🔴 Gap 3: Audio and Video Treated Separately

Gap:

Audio spoofing and video manipulation are studied in isolation, ignoring cross-modal inconsistencies.

Our Approach:

Explicit audio–video cross-verification through joint score fusion, enabling detection of partial manipulations.

🔴 Gap 4: Lack of Explainability

Gap:

Most methods report accuracy only, offering black-box predictions.

Our Approach:

Built-in explainability using Grad-CAM, audio heatmaps, and temporal visualizations, delivered as a forensic report.

🔴 Gap 5: No Deployment Focus

Gap:

Existing works remain offline academic prototypes with no real-world integration.

Our Approach:

API-driven, modular system designed for web, mobile, and enterprise deployment.

Try Pitch

System Requirements

1. Hardware Requirements Development / Training Environment

- GPU: NVIDIA T4 / RTX 3060 or higher (minimum 8 GB VRAM; recommended 16 GB+)
- CPU: Quad-core Intel i5 / Ryzen 5 or higher
- RAM: Minimum 16 GB (Recommended: 32 GB for multi-modal processing)
- Storage:
  - Minimum: 200 GB free space
  - Recommended: 1 TB SSD (for datasets + features)
- Internet: Stable broadband connection for model downloads

Deployment Environment

- CPU server or GPU server (depending on inference speed requirement)
- 8 GB RAM minimum
- SSD storage for fast media access
- Load balancer for production use

2. Software Requirements

Operating System Windows 10/11 (development only)

Programming Environment

- **Python 3.9+**
- **CUDA Toolkit 11+**
- **cuDNN compatible with CUDA version**

Libraries & Frameworks

Core AI

- **PyTorch**
- **TorchVision**
- **timm (EfficientNet)**
- **Transformers (Wav2Vec2 / AV-HuBERT)**

Computer Vision

- **OpenCV**
- **MTCNN / InsightFace**
- **NumPy**

Audio Processing: Librosa, SciPy

Try Pitch

System Requirements

3. Model Requirements

The system requires pretrained or trained versions of:

- EfficientNet-B4 (spatial branch)
- Temporal Convolutional Network
- Audio CNN + TCN
- Lip-sync model (SyncNet or AV-HuBERT)
- Fusion MLP model

All models must be stored in serialized checkpoint format (.pth / .pt).

4. Data Requirements

- Supported Formats:
  - Video: MP4, AVI, MOV
  - Audio: WAV, MP3
- Input Constraints:
  - Minimum resolution: 224×224
  - Minimum duration: 1 second, Maximum duration: configurable
- Dataset Requirements:
  - Labeled real vs fake media
  - Balanced class distribution preferred

5. Performance Requirements

- Frame processing latency: < 50 ms per frame
- Full video inference time:
  - GPU: < 5 seconds (short clips)
  - CPU: < 20 seconds
- Accuracy target:
  - AUC ≥ 0.90 on benchmark dataset
- Throughput:
  - ≥ 10 videos/minute (GPU inference server)

Try Pitch

System Requirements

6. Functional Requirements

The system must:

- Accept video/audio inputs
- Extract frames and audio signals
- Detect faces
- Compute spatial, temporal, audio, and physiological features
- Perform multi-modal fusion
- Output authenticity score
- Generate explainability report

7. Non-Functional Requirements Reliability

- Must handle corrupted files gracefully
- Must resume processing after interruption

Scalability

- Should support horizontal scaling via container orchestration

Security

- Must sanitize uploaded files
- Must prevent malicious media execution

Usability

- API must return structured JSON results
- Interface must show confidence and explanation

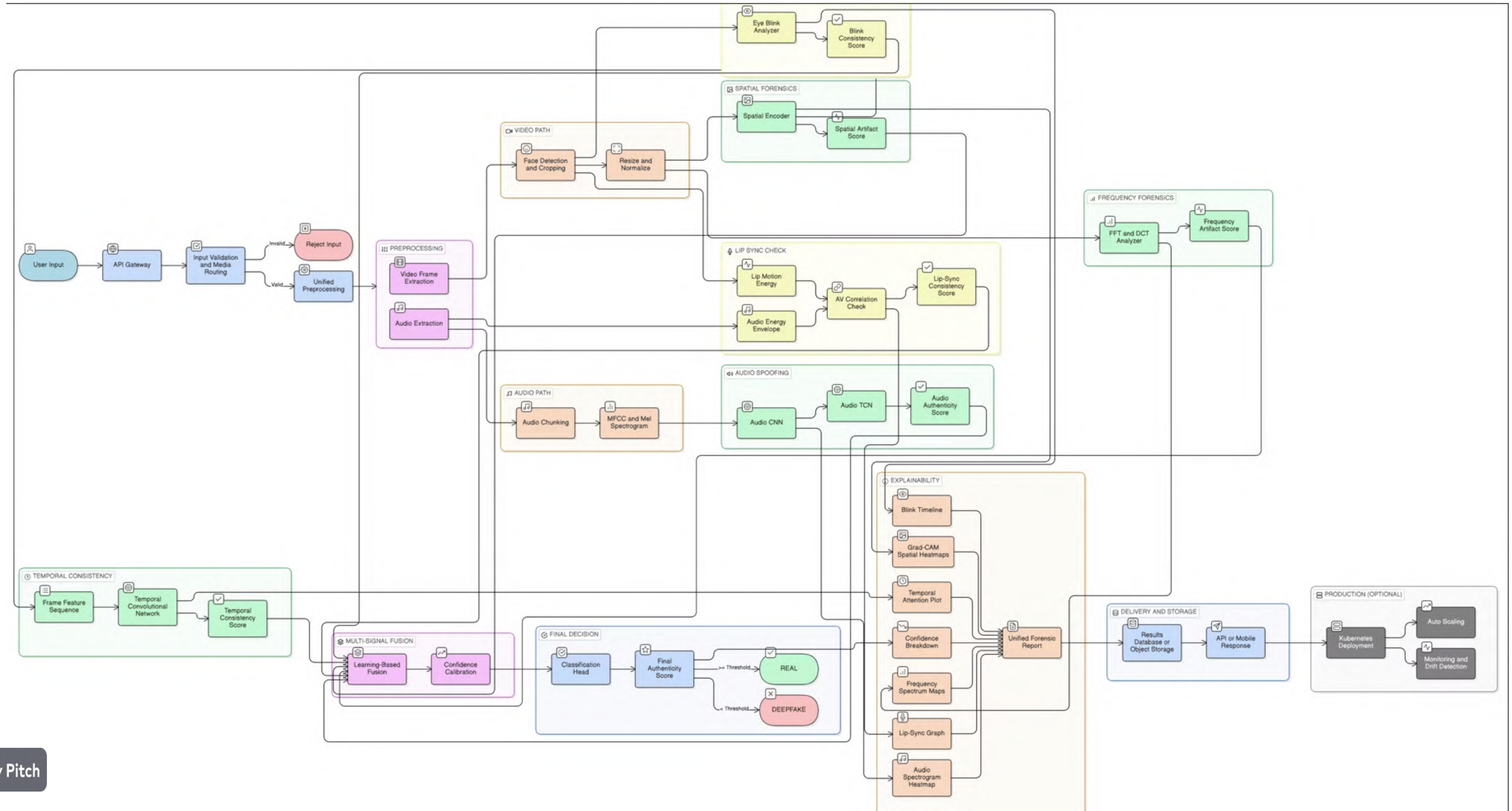
8. Deployment Requirements

Production deployment must support:

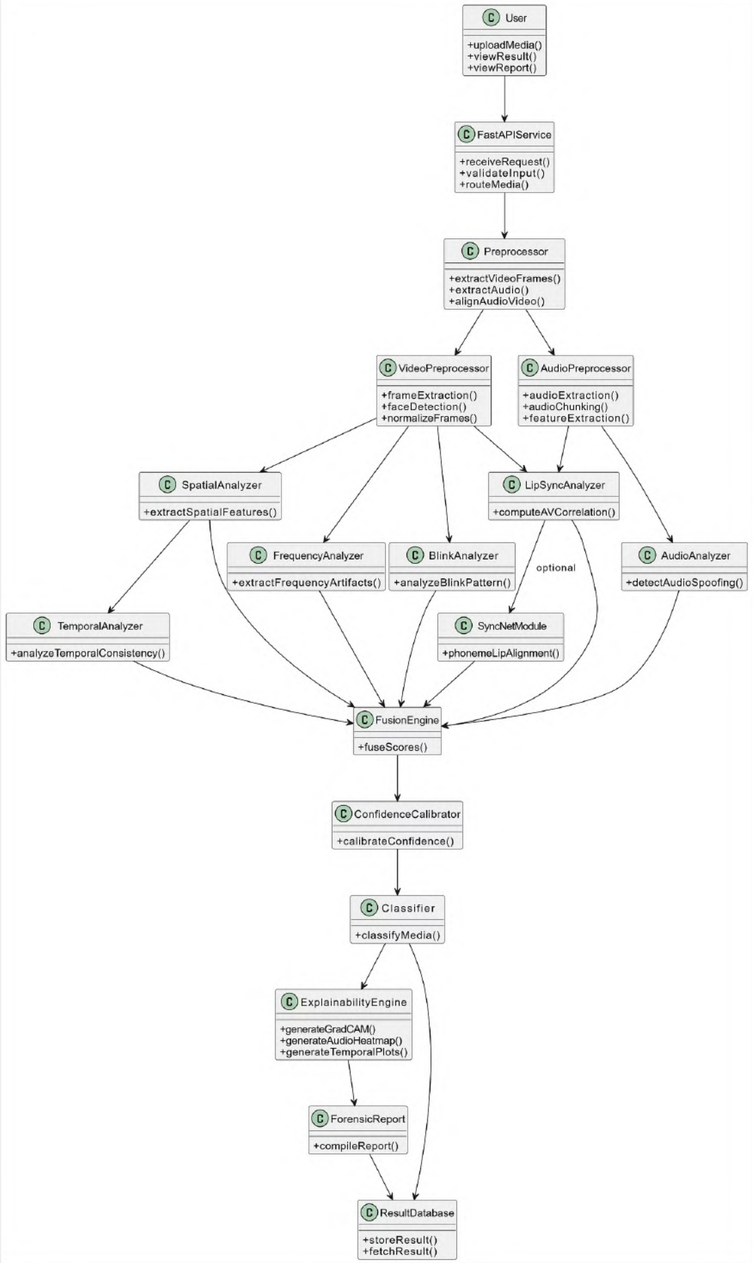
- Docker containers
- Kubernetes orchestration
- Auto-scaling
- Logging & monitoring
- Model versioning

Try Pitch

Architecture Design



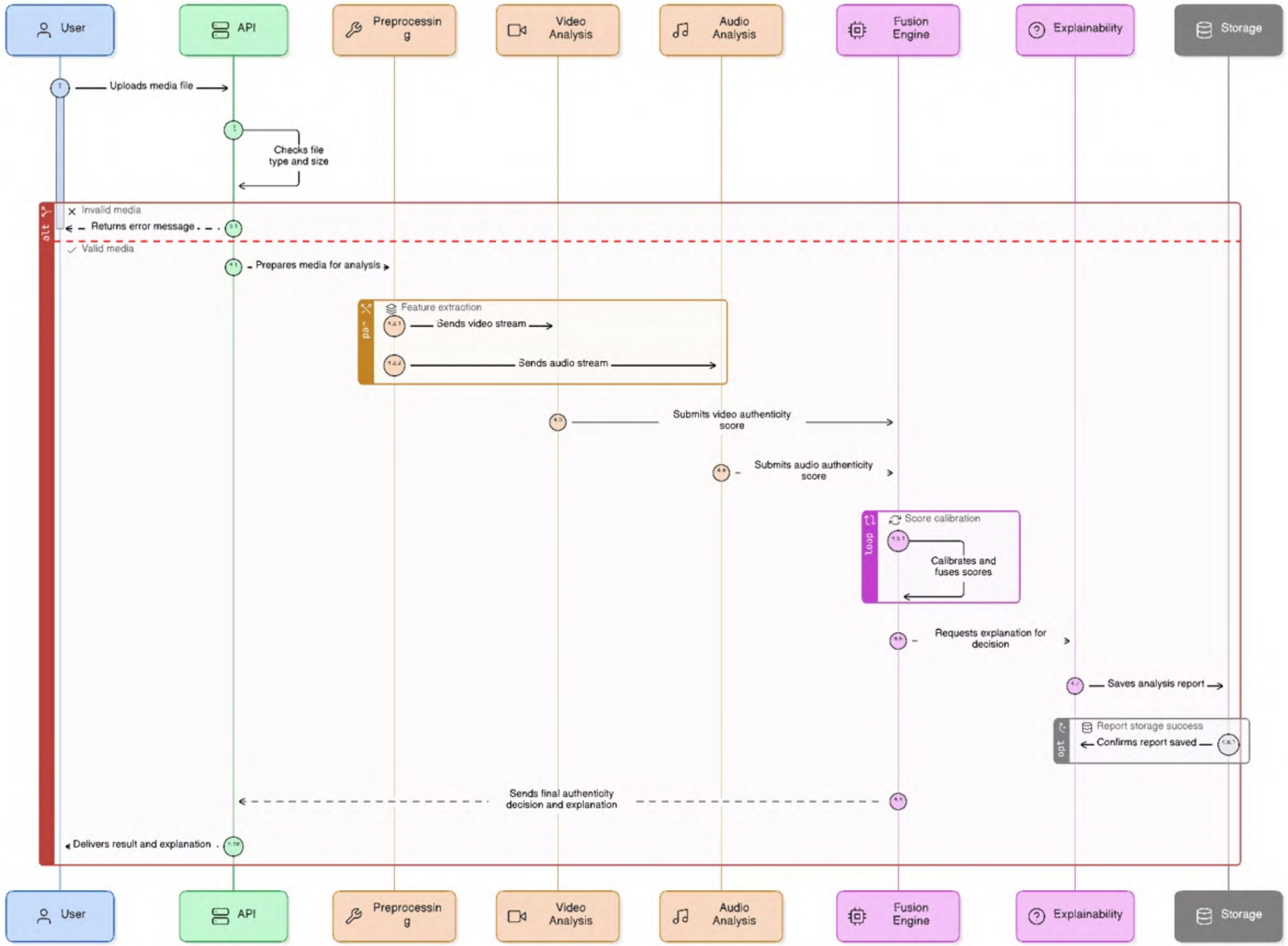
Try Pitch



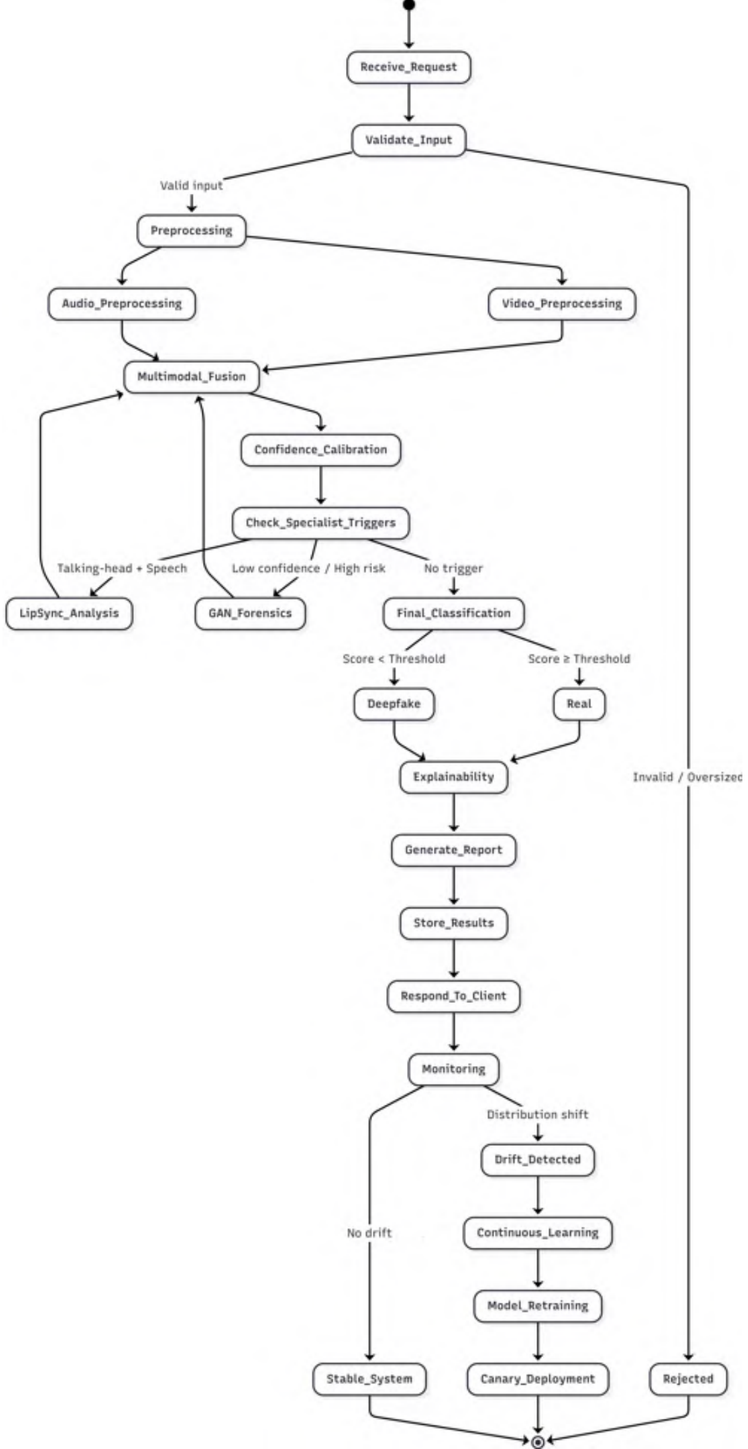
UML Diagrams - Class Diagram

Try Pitch

UML Diagrams - Sequence Diagram



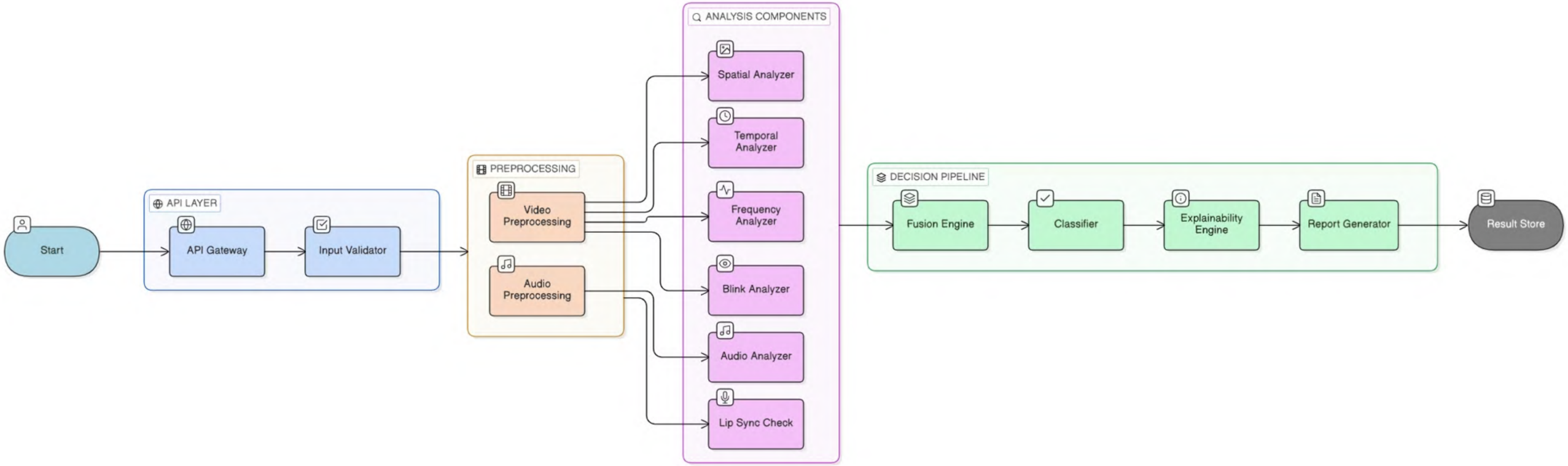
Try Pitch



UML Diagrams - State Diagram

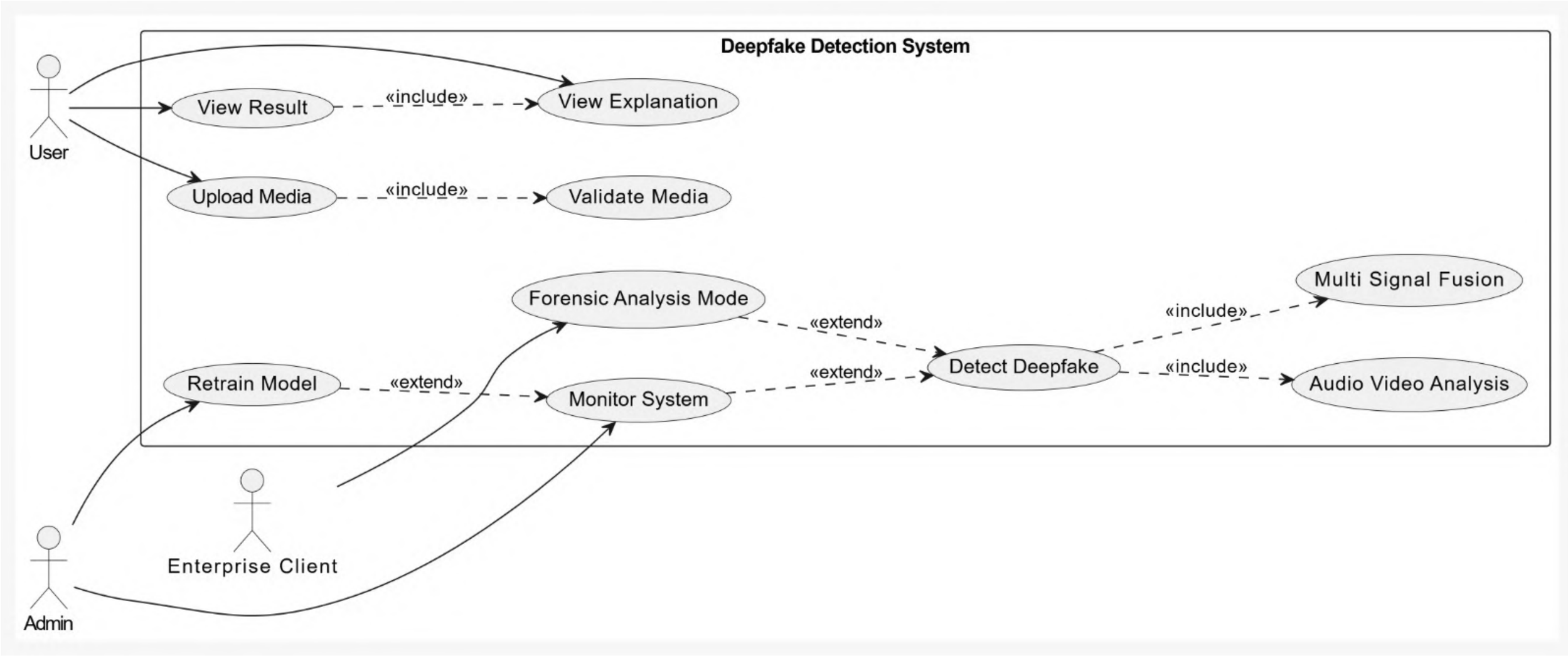
Try Pitch

UML Diagrams - Component Diagram

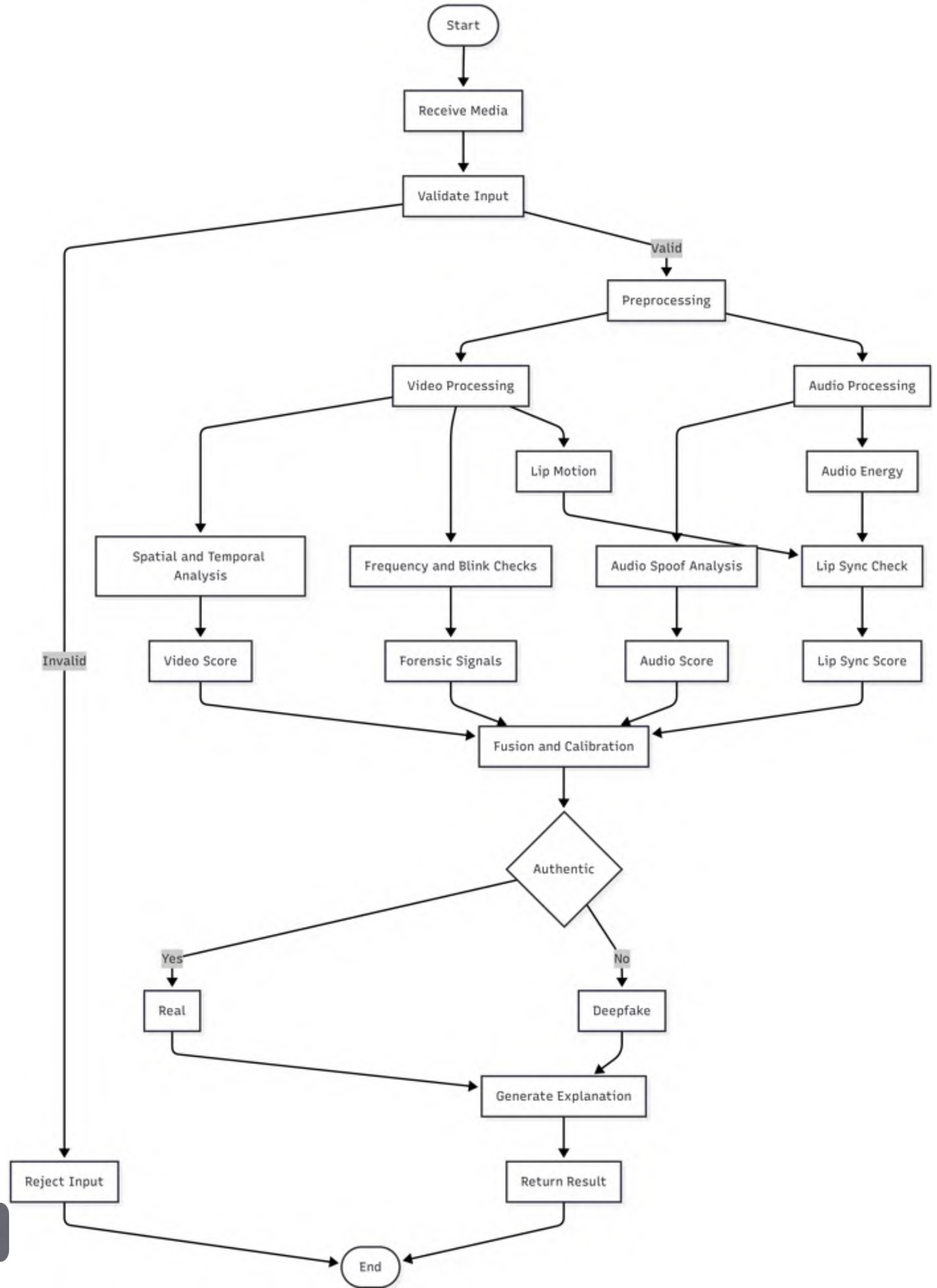


Try Pitch

UML Diagrams - Use Case Diagram



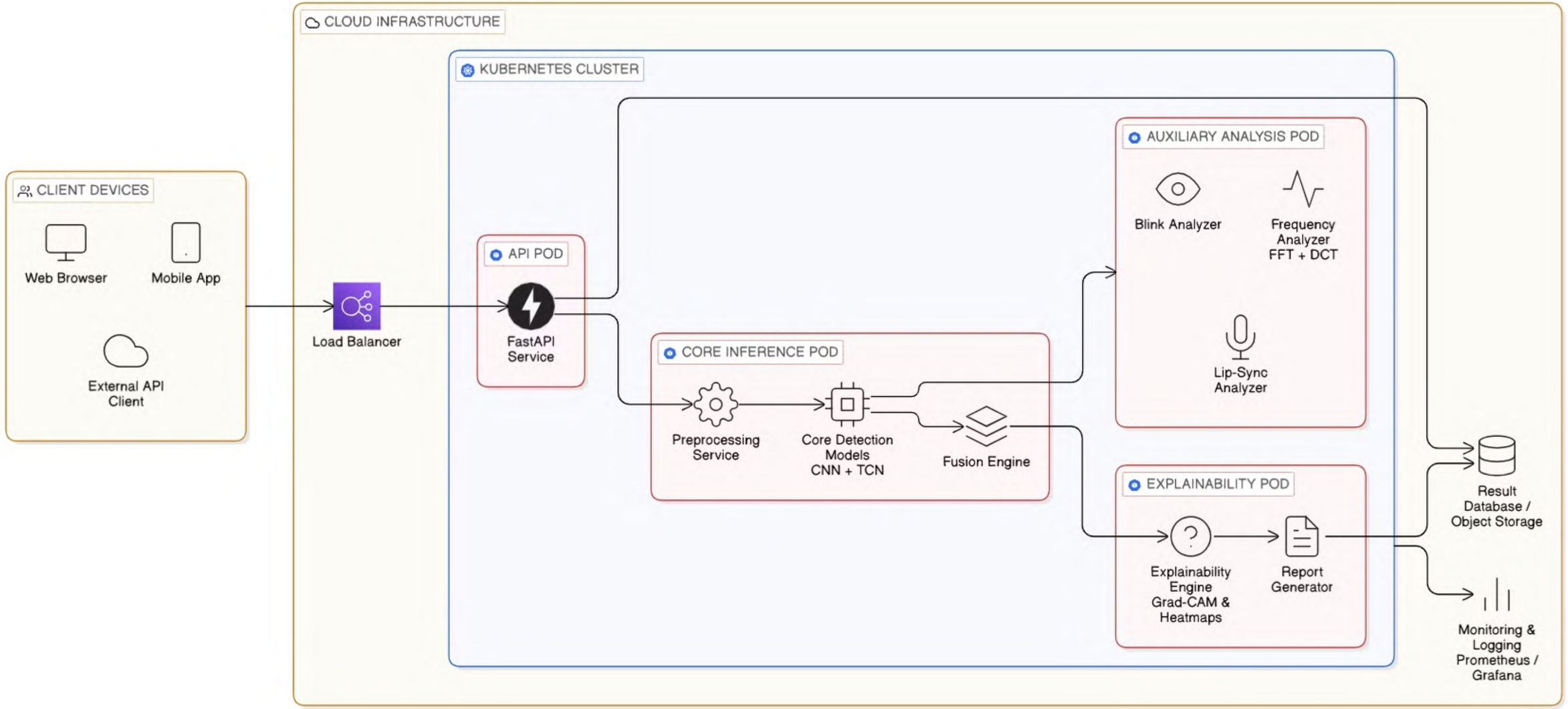
Try Pitch



UML Diagrams - Activity Diagram

Try Pitch

UML Diagrams - Deployment Diagram



Try Pitch

Core Modules

Module 1: Input Handler & Validation Layer

Purpose: Handle and validate incoming media inputs from multiple sources.

Responsibilities:

- Accept video/audio uploads
- Detect file format and type
- Validate size, duration, codec
- Route media to correct pipeline
- Reject corrupted or unsupported inputs

Technologies: FastAPI, Python MIME, FFmpeg, OpenCV

✅ **Module 2: Unified Preprocessing Engine**

Purpose: Prepare multimedia data for analysis across all forensic branches.

Responsibilities:

- Video frame extraction (15–30 FPS)
- Audio extraction
- Frame normalization (224×224)
- Data buffering for batch processing,Metadata extraction

Technologies: OpenCV, FFmpeg, NumPy, PIL

✅ **Module 3: Face Processing Module**

Purpose: Detect and isolate facial regions for forensic analysis.

Responsibilities:

- Face detection (MTCNN)
- Face cropping
- Landmark detection
- Face alignment
- Region tracking across frames

Technologies: MTCNN, Dlib, OpenCV

✅ **Module 4: Spatial Forensics Module (Core)**

Purpose: Detect visual manipulation artifacts in facial frames.

Responsibilities:

- Extract spatial features
- Detect blending artifacts
- Identify texture inconsistencies
- Compute spatial authenticity score

Model: EfficientNet-B4 CNN

Try Pitch

Core Modules

Module 5: Frequency Forensics Module

Purpose: Identify manipulation artifacts invisible in pixel space.

Responsibilities:

- FFT transformation
- DCT analysis
- Frequency band anomaly detection
- Compression artifact analysis

Technologies: NumPy FFT, SciPy DCT

✅ **Module 6: Temporal Consistency Module (Core)**

Purpose: Detect unnatural motion patterns across frames.

Responsibilities:

- Build frame feature sequence
- Capture motion consistency
- Detect frame transition anomalies
- Model long-range temporal patterns

Model: Temporal Convolutional Network (TCN)

✅ **Module 7: Audio Authenticity Module (Core)**

Purpose: Detect synthetic or manipulated speech.

Responsibilities:

- Extract MFCC and Mel spectrogram
- Analyze speech spectral structure
- Model temporal speech patterns
- Detect vocoder artifacts

Model: CNN + Audio TCN

✅ **Module 8: Physiological Signal Analyzer**

Purpose: Validate biological realism of subject.

Responsibilities:

- Blink detection
- Blink frequency analysis
- Eye aspect ratio computation
- Heart-rate (rPPG) signal estimation
- Physiological consistency scoring

Try Pitch

Core Modules

Module 9: Lip-Sync Verification Module Lightweight Layer (Default)

Purpose: Quickly verify audio–visual consistency.

Responsibilities:

- Lip motion energy extraction
- Audio energy envelope extraction
- Correlation computation

Advanced Layer (Triggered Mode)

Purpose: Perform precise phoneme–lip alignment verification.

Trigger Conditions:

- Low lightweight score
- Talking head detected
- High-risk mode enabled

Models: SyncNet / AV-HuBERT

✅ **Module 10: Multi-Signal Fusion Engine**

Purpose: Combine evidence from all modalities into a unified decision.

Responsibilities:

- Aggregate modality scores
- Learn modality importance weights
- Handle missing modalities
- Produce unified authenticity logit

Models: MLP Fusion Network / XGBoost Ensemble

Module 11: Confidence Calibration & Decision Module

Purpose: Generate reliable probability estimates and final classification.

Responsibilities:

- Probability calibration
- Threshold decision logic
- Risk-aware decision adjustment
- Output final authenticity score

Methods: Temperature scaling, logistic calibration

Try Pitch

Core Modules

Module 12: Explainability & Forensic Report Engine

Purpose: Provide interpretable evidence for predictions.

Responsibilities:

- Spatial heatmaps (Grad-CAM)
- Frequency maps
- Temporal attention visualization
- Spectrogram heatmaps
- Blink timeline plots
- Lip-sync graphs
- Confidence breakdown charts

Output:

Unified forensic report

Module 13: Output Delivery Layer

Purpose: Deliver results to client systems.

Responsibilities:

- JSON API response
- Result storage
- Evidence linking
- Dashboard visualization

Technologies: FastAPI, Object Storage, Database

Module 14: Deployment & Monitoring Layer

Purpose: Enable scalable, production-ready deployment.

Responsibilities:

- Container orchestration
- Auto scaling
- Performance monitoring
- Drift detection
- Logging and alerting

Technologies: Kubernetes, Prometheus, Grafana

Try Pitch

Implementation

Implemented a multi-modal deepfake detection pipeline that preprocesses DFDC videos to extract spatial, motion, audio, and physiological features using GPU-accelerated models. Each modality is modeled with dedicated neural networks and trained separately for authenticity prediction. The learned embeddings are combined using a late-fusion MLP to produce the final deepfake classification. The system includes checkpointing, mixed-precision training, and robust data processing for efficient large-scale experimentation.

```
[2026-02-10 08:36:53] face Epoch 1 finished in 37.0s loss 0.3461 train_auc 0.5528
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:37:00] Eval face: AUC 0.5691
[2026-02-10 08:37:00] face checkpoint saved to /content/drive/MyDrive/dfdc_train_ckpts/face_best.pth
/tmp/ipython-input-602814789.py:293: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:37:36] face Epoch 2 finished in 36.1s loss 0.2086 train_auc 0.7616
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:37:43] Eval face: AUC 0.7470
[2026-02-10 08:37:43] face checkpoint saved to /content/drive/MyDrive/dfdc_train_ckpts/face_best.pth
/tmp/ipython-input-602814789.py:293: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
[2026-02-10 08:39:54] Eval face: AUC 0.9250
[2026-02-10 08:39:54] face checkpoint saved to /content/drive/MyDrive/dfdc_train_ckpts/face_best.pth
/tmp/ipython-input-602814789.py:293: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:40:31] face Epoch 6 finished in 37.1s loss 0.1230 train_auc 0.9546
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:40:38] Eval face: AUC 0.9284
[2026-02-10 08:40:38] face checkpoint saved to /content/drive/MyDrive/dfdc_train_ckpts/face_best.pth
[2026-02-10 08:40:38] face best val AUC = 0.9284
[2026-02-10 08:40:38] Training Motion encoder ...
/tmp/ipython-input-602814789.py:293: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
```

Try Pitch

Implementation

```
[2026-02-10 08:41:15] motion Epoch 1 finished in 36.6s loss 0.3968 train_auc 0.5255
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:41:22] Eval motion: AUC 0.4918
[2026-02-10 08:41:22] motion checkpoint saved to /content/drive/MyDrive/dfdc_train_ckpts/motion_best.pth
/tmp/ipython-input-602814789.py:293: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:41:58] motion Epoch 2 finished in 36.3s loss 0.2923 train_auc 0.5340
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
```

```
[2026-02-10 08:44:11] Eval motion: AUC 0.4560
/tmp/ipython-input-602814789.py:293: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:44:47] motion Epoch 6 finished in 35.5s loss 0.2824 train_auc 0.5495
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:44:54] Eval motion: AUC 0.4826
[2026-02-10 08:44:54] motion best val AUC = 0.4918
```

```
[2026-02-10 08:44:54] Training Audio encoder ...
/tmp/ipython-input-602814789.py:293: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:45:29] audio Epoch 1 finished in 35.2s loss 0.4079 train_auc 0.5021
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:45:36] Eval audio: AUC 0.4386
[2026-02-10 08:45:36] audio checkpoint saved to /content/drive/MyDrive/dfdc_train_ckpts/audio_best.pth
/tmp/ipython-input-602814789.py:293: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:46:12] audio Epoch 2 finished in 36.3s loss 0.2442 train_auc 0.5582
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
[2026-02-10 08:49:04] audio Epoch 6 finished in 35.8s loss 0.2313 train_auc 0.6463
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:49:11] Eval audio: AUC 0.3250
[2026-02-10 08:49:11] audio best val AUC = 0.4386
```

Try Pitch

Implementation

```
[2026-02-10 08:49:11] Training Phys encoder ...
/tmp/ipython-input-602814789.py:293: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:49:47] phys Epoch 1 finished in 36.0s loss 0.5587 train_auc 0.5092
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:49:53] Eval phys: AUC 0.4949
[2026-02-10 08:49:53] phys checkpoint saved to /content/drive/MyDrive/dfdc_train_ckpts/phys_best.pth
/tmp/ipython-input-602814789.py:293: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:50:29] phys Epoch 2 finished in 36.1s loss 0.3778 train_auc 0.5003
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
```

```
[2026-02-10 08:53:21] phys Epoch 6 finished in 35.7s loss 0.2833 train_auc 0.5315
/tmp/ipython-input-602814789.py:331: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:53:27] Eval phys: AUC 0.4580
[2026-02-10 08:53:27] phys best val AUC = 0.4949
[2026-02-10 08:53:27] Per-stream training complete. Results so far: {'face': np.float64(0.9284442285899304), 'motion': np.float64(0.49178403755868544), 'audio': np.float64(0.43
[2026-02-10 08:53:27] Loaded face weights from /content/drive/MyDrive/dfdc_train_ckpts/face_best.pth
[2026-02-10 08:53:27] Loaded motion weights from /content/drive/MyDrive/dfdc_train_ckpts/motion_best.pth
[2026-02-10 08:53:27] Loaded audio weights from /content/drive/MyDrive/dfdc_train_ckpts/audio_best.pth
[2026-02-10 08:53:27] Loaded phys weights from /content/drive/MyDrive/dfdc_train_ckpts/phys_best.pth
[2026-02-10 08:53:27] Fusion model created. Training fusion head only.
/tmp/ipython-input-602814789.py:440: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
  scaler_f = torch.cuda.amp.GradScaler(enabled=USE_AMP)
/tmp/ipython-input-602814789.py:449: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:54:10] Fusion Epoch 1 done in 36.3s  val_auc 0.8540
```

```
[2026-02-10 08:57:02] Saved fusion_best.pth
/tmp/ipython-input-602814789.py:449: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=USE_AMP):
[2026-02-10 08:57:44] Fusion Epoch 6 done in 36.5s  val_auc 0.9635
[2026-02-10 08:57:44] Saved fusion_best.pth
[2026-02-10 08:57:44] FINAL RESULTS (AUC):
Face only       0.9284
Motion only     0.4918
Audio only      0.4386
Physiology only 0.4949
Fusion (all)    0.9635
[2026-02-10 08:57:44] Training complete. Checkpoints and results saved to: /content/drive/MyDrive/dfdc_train_ckpts
```

Try Pitch

Current Project Status

- ☐ Input Handler & Validation Layer
- ☑ Unified Preprocessing Engine
- ☑ Face Processing Module
- ☑ Spatial Forensics Module
- ☐ Frequency Forensics Module
- ☑ Temporal Consistency Module
- ☑ Audio Authenticity Module
- ☑ Physiological Signal Analyzer
- ☐ Lip-Sync Verification Module
- ☑ Multi-Signal Fusion Engine
- ☐ Confidence Calibration & Decision Module
- ☐ Explainability & Forensic Report Engine
- ☐ Output Delivery Layer
- ☐ Deployment & Monitoring Layer

Try Pitch

Thank You

- Team Yuktha

Gaddam Koushik - 22071A6916

Paitha Kommineni - 22071A6934

K.N.R. Rishik - 22071A6938

Guide - Mrs. V Manjula

Try Pitch

Pitch

Want to make a presentation like this one?

Start with a fully customizable template, create a beautiful deck in minutes, then easily share it with anyone.

Create a presentation (It's free)